

Recette & Avis

Suivi de projet - TI616 Numérique Durable

Groupe 2 - EFREI Paris 2025-2026

1. Objectif du projet

Recette & Avis est une plateforme de consultation de recettes de cuisine enrichie d'un système d'évaluation communautaire. Elle permet aux utilisateurs de parcourir des recettes fiables, de les noter et de partager leurs avis, avec un accent fort sur la sobriété numérique et l'éco-conception.

Problème résolu : il est souvent difficile d'évaluer rapidement la fiabilité d'une recette en ligne et de trouver des retours concrets d'autres cuisiniers. Notre plateforme centralise recettes vérifiées, notes et commentaires pour guider les choix des utilisateurs.

2. Stack technique

Couche	Technologie	Justification Green IT
Backend	Node.js + Express	Léger, sans surcharge, démarrage rapide, faible empreinte mémoire
Base de données	SQLite	Fichier unique, aucun serveur dédié, sobre en ressources, parfait pour un MVP
Frontend	HTML5 sémantique + CSS + JS minimal	Zéro dépendance lourde, chargement rapide, pleinement maîtrisé
Hébergement	Render / Railway (en cours)	Hébergement léger, sans surpuissance inutile, tier gratuit disponible
Versionnement	Git + GitHub	Standard collaboratif, obligatoire selon le sujet

3. Fonctionnalités implémentés (ou en développement)

Authentification & Profils

- Inscription avec validation email et force du mot de passe
- Connexion / déconnexion avec gestion de session
- Édition du profil (nom, email, mot de passe)
- Suppression du compte utilisateur

Recettes

- Affichage de la liste des recettes (titre, catégorie, temps de préparation, note moyenne)
- Affichage du détail d'une recette (ingrédients, étapes, commentaires, notes)
- Interface d'administration pour créer, modifier et supprimer des recettes
- Filtrage par catégorie

Notes & Commentaires

- Ajout d'une note (1 à 5 étoiles), une seule note par utilisateur par recette
- Calcul et affichage de la note moyenne
- Ajout, modification et suppression d'un commentaire
- Modération des commentaires par l'administrateur

Interface Admin

- Gestion des recettes (CRUD complet)
- Gestion des utilisateurs (liste, modification, suppression)
- Modération des commentaires

4. Structure de la base de données

```
utilisateurs (id, nom, email, mot_de_passe, role, date_creation)
recettes (id, titre, description, ingredients, etapes, temps_preparation,
categorie, image)
commentaires (id, #recette_id, #utilisateur_id, contenu, date_commentaire)
notes (id, #recette_id, #utilisateur_id, valeur)
```

Catégories disponibles : Entrée, Plat principal, Dessert, Végétarien, Rapide (<30 min), Autre.

5. Ce qui a déjà été produit

Livrable	État
Schéma BDD (db.sql / schema SQLite)	Finalisé
Diagramme de cas d'utilisation	Finalisé (Livrable 1)
Diagramme de classes / MCD	Finalisé (Livrable 1)
Diagramme de séquence (ajout commentaire)	Finalisé (Livrable 1)
Wireframes / maquettes des pages principales	Finalisés (Livrable 1)
Backend Node.js + Express (routes auth, recettes, commentaires, notes)	En cours
Frontend HTML/CSS (pages principales)	En cours
Interface admin (CRUD recettes + utilisateurs)	En cours
README structuré	En cours (à compléter avec URL déployée)

6. Ce qu'il reste à faire

Développements

- Finalisation du CRUD recettes côté admin (modification + suppression)
- Validation serveur sur tous les formulaires (recettes, commentaires, inscription)
- Pagination des listes (recettes, utilisateurs en admin)
- Gestion des erreurs HTTP (404, 403, 500) avec pages dédiées
- Déploiement sur Render ou Railway avec variables d'environnement (.env)
- Lazy loading des images sur la page de détail recette

Optimisations Green IT à mettre en place

- Minification du CSS (outil : cssnano ou minification manuelle)
- Compression et conversion des images en WebP (max 300 Ko)
- Vérification : nombre de requêtes HTTP < 15 par page
- Ajout des attributs alt sur toutes les images
- Vérification des contrastes texte/fond (ratio $\geq 4,5:1$)
- Suppression des dépendances npm non utilisées

Tests & Qualité

- Tableau de scénarios de tests fonctionnels à remplir (≥ 8 scénarios)
- Analyse Lighthouse sur les 3 pages principales (objectif : score > 80)
- Mesures EcoIndex et Website Carbon Calculator avant/après optimisations
- Vérifications de sécurité : mots de passe hashés, pas de credentials dans le dépôt, protection injections SQL

7. Approche éco-conception

Pratique	Détail
Polices système	Utilisation de la font-stack système (system-ui, sans-serif)
JS minimal	JavaScript limité aux interactions strictement nécessaires (notation étoiles, validation formulaire)
Images optionnelles	L'image d'une recette est optionnelle ; si présente, compressée en WebP, max 300 Ko
Lazy loading	Images chargées uniquement à la demande (loading="lazy") sur la page de détail
SQLite	Base de données fichier unique, aucun serveur dédié, consommation minimale
HTML sémantique	Balises natives (<main>, <article>, <nav>, <footer>), pas de div inutiles
Données sobres	Seules les colonnes strictement nécessaires sont stockées et récupérées (pas de SELECT *)
Pagination	Listes paginées pour éviter de charger des centaines de recettes d'un coup

8. Points forts / Points faibles

Points forts

- Approche Green IT cohérente dès la conception.
- Architecture sobre et réaliste pour le périmètre du projet (Node.js + SQLite)
- Entité métier pertinente (recettes + notes + commentaires) avec contrainte d'unicité sur les notes
- Livrable 1 complet : diagrammes UML, MCD, wireframes, user stories, backlog

Points faibles / Points d'attention

- Déploiement pas encore en ligne, à finaliser avant la présentation
- Variables d'environnement à externaliser dans un .env (connexion BDD actuellement en dur en local)
- Pas encore de pagination côté backend (à implémenter pour les listes recettes et utilisateurs)
- Tests fonctionnels non encore documentés formellement, tableau à compléter
- Mesures Green IT à réaliser sur la version déployée (impossible en local pur)